



Institución:

Instituto Tecnológico de Tepic

Curso:

Programación Web

Unidad 5

Título:

Investigación

Autor:

Isais Nava Gandhi Antonio – 234005666

Docente:

Irving Yair Nava Hernández

Fecha:

20 – 07 – 2026

Lado del cliente

¿Qué es?

El "lado del cliente" hace referencia a todo lo que en una aplicación web se muestra o tiene lugar en el cliente (dispositivo del usuario final). Esto incluye lo que ve el usuario, como texto, imágenes y el resto de la interfaz de usuario, junto con cualquier acción que una aplicación lleve a cabo en el navegador del usuario.

Lenguajes más utilizados

Los lenguajes más utilizados del lado del cliente son **JavaScript**, **HTML** y **CSS**. Estos se ejecutan directamente en el navegador del usuario. JavaScript aporta la lógica y dinamismo, mientras que HTML estructura el contenido y CSS se encarga del diseño visual y la maquetación.

Tecnologías y herramientas comunes

Las tecnologías y herramientas del lado del cliente son aquellas que se ejecutan directamente en el navegador web del usuario. Se dividen en los tres lenguajes web fundamentales (HTML, CSS, JavaScript), además de frameworks modernos como React, Vue.js, y herramientas de compilación como Webpack.

Ventajas

- **Interactividad y velocidad:** Al delegar la interfaz a tecnologías como JavaScript, el usuario obtiene respuestas visuales inmediatas y fluidez sin necesidad de enviar peticiones constantes al servidor.
- **Ahorro de recursos en el servidor:** Como el dispositivo del usuario ejecuta la lógica local y el procesamiento, se minimiza el uso de la memoria y capacidad de procesamiento central.
- **Experiencia similar a una app nativa:** Permite la carga asíncrona de datos (mediante AJAX), actualizando solo partes específicas de la web.
- **Menos consumo de ancho de banda:** Solo se transfieren los datos y plantillas necesarias, evitando enviar páginas HTML completas desde el servidor en cada interacción.

Desventajas

- **Problemas de SEO:** Los buscadores pueden tener dificultades para indexar contenido generado dinámicamente con JavaScript.

- **Vulnerabilidades de seguridad:** El código fuente es visible para cualquiera, lo que facilita la manipulación de datos y ataques maliciosos.
- **Carga inicial lenta:** El navegador debe descargar todos los scripts de JavaScript antes de poder mostrar la página completa.
- **Dependencia del hardware:** El rendimiento final de la web depende de la potencia y memoria del dispositivo del usuario.
- **Inconsistencia entre navegadores:** El código puede ejecutarse de forma diferente o fallar según el navegador o versión que use el cliente.

Seguridad

La seguridad del lado del cliente protege el código (HTML, CSS y JavaScript) que se ejecuta en el navegador frente a la inyección y manipulación de datos. Sus técnicas incluyen la validación de entradas, la aplicación de directivas CSP (Política de Seguridad de Contenido) y el monitoreo de scripts de terceros.

Ubicación

Navegador (Lado del Cliente)

- El código se descarga y se ejecuta en el **dispositivo del usuario**.
- Utiliza principalmente **JavaScript, HTML y CSS**.
- Es ideal para la **interactividad inmediata**, animaciones y validaciones rápidas.
- Reduce la carga del servidor al procesar datos localmente.
- Es **inseguro por naturaleza** porque el usuario tiene acceso total al código.

Lado del Servidor

¿Qué es?

El "lado del servidor" se refiere a todo lo que ocurre en el servidor, en lugar de en el cliente. En el pasado, casi toda la lógica empresarial se ejecutaba en el lado del servidor, y esto incluía la representación de páginas web dinámicas, la interacción con las bases de datos, la autenticación de identidades y las notificaciones push.

Lenguajes más utilizados

Los lenguajes más utilizados del lado del servidor (backend) procesan datos, gestionan bases de datos y conectan la interfaz con la lógica de la aplicación. Los líderes actuales son **JavaScript** (a través de **Node.js**), **PHP**, **Python**, **Java** y **C#** (con ASP.NET).

Tecnologías y herramientas comunes

Las tecnologías del lado del servidor (backend) procesan las peticiones web, gestionan la lógica de negocio y se conectan a las bases de datos.

- Lenguajes de programación: JavaScript, Python, PHP, Java, etc.
- Frameworks Backend: Express.js, Django y Flask, Laravel
- Servidores web: Apache HTTP Server, Nginx
- Bases de datos: Relacionales (SQL), No Relacionales (NoSQL)
- Contenedores y despliegues: Docker y Kubernetes.

Ventajas

- **Optimización para motores de búsqueda (SEO):** Al realizar el *Server-Side Rendering* (SSR), el servidor entrega contenido completamente estructurado en HTML. Esto facilita la indexación por parte de los *crawlers* de Google u otros motores.
- **Velocidad de carga inicial:** El usuario recibe una página completamente renderizada y lista para visualizar, lo que disminuye el tiempo de espera.
- **Seguridad:** Las operaciones críticas, como el manejo de contraseñas, claves API y transacciones de pago, ocurren en el backend y no en el dispositivo del cliente. Esto previene la manipulación o la inyección de código malicioso.

- **Menor dependencia del dispositivo:** El servidor asume la mayor parte del trabajo pesado (cálculos, consultas a bases de datos). Los teléfonos o computadoras de los usuarios requieren menos potencia de hardware para visualizar el sitio.
- **Centralización y consistencia:** Almacena y administra los datos en un solo lugar centralizado, garantizando que todos los usuarios accedan a la misma información en tiempo real.

Desventajas

- **Mayor costo y carga en el servidor:** El backend procesa cada solicitud de cada usuario individual. Esto requiere servidores más potentes y costosos para evitar caídas ante picos de tráfico masivos.
- **Interacciones más lentas (Latencia):** Cada clic o acción que requiera nuevos datos obliga a realizar un viaje completo de ida y vuelta (*round-trip*) al servidor, lo que puede pausar la experiencia del usuario.
- **Recarga completa de pantalla:** A diferencia de las aplicaciones del lado del cliente, las transiciones entre páginas suelen requerir que el navegador parpadee o se recargue por completo para mostrar el nuevo HTML.
- **Mayor tiempo para el primer byte (TTFB):** Aunque el usuario ve la página rápido una vez que llega, el servidor tarda más tiempo en empezar a enviar los primeros datos porque primero debe procesar la base de datos y armar el HTML. [[1](#)]
- **Desarrollo más complejo:** Requiere configurar y mantener infraestructuras de backend, gestionar servidores, bases de datos y balanceadores de carga, lo que eleva la dificultad técnica.
- **Dependencia total de la conexión:** Si el usuario pierde el acceso a internet o el servidor falla, la aplicación web deja de funcionar por completo, perdiendo cualquier capacidad de uso sin conexión (*offline*).

Seguridad

La seguridad del lado del servidor es **la barrera más fuerte** de una aplicación porque mantiene los datos críticos y el código lógico totalmente fuera del alcance del usuario. Al centralizar el control en el backend, se evita que los atacantes manipulen la aplicación modificando el código desde el navegador.

- Ocultamiento de datos sensibles

- Validación de datos absoluta
- Control de acceso robusto
- Mitigación de ataques comunes
- Auditoría y registros seguros

Ubicación de ejecución

La ubicación de ejecución del lado del servidor es el **backend**, es decir, cualquier entorno de computación remoto que esté completamente separado del navegador web del usuario. El código no se ejecuta en el dispositivo de la persona que visita la página, sino en la infraestructura física o virtual del proveedor.

Cuadro Comparativo

Criterio	Client-side (Lado del Cliente)	Server-side (Lado del Servidor)
Descripción general	• Se refiere a todo lo que se muestra o sucede en el dispositivo del usuario final (navegador). • Incluye la interfaz visual, texto, imágenes y las acciones que la aplicación ejecuta en el navegador.	• Se refiere a todo lo que ocurre en el servidor, en lugar de en el dispositivo del cliente. • Incluye la lógica empresarial, la representación de páginas dinámicas, la interacción con bases de datos y la autenticación.
Lenguajes más utilizados	• JavaScript • HTML • CSS	• JavaScript (Node.js) • PHP • Python • Java • C# (ASP.NET)
Tecnologías y herramientas	• HTML, CSS y JavaScript (lenguajes web fundamentales) • Frameworks: React, Vue.js • Herramientas de compilación: Webpack	• Frameworks backend: Express.js, Django, Flask, Laravel • Servidores web: Apache HTTP Server, Nginx • Bases de datos: relacionales (SQL) y no relacionales (NoSQL) • Contenedores y despliegue: Docker y Kubernetes
Ventajas	• Interactividad y velocidad de respuesta sin peticiones constantes al servidor • Ahorro de recursos en el servidor • Experiencia similar a una app nativa (carga asíncrona vía AJAX) • Menor consumo de ancho de banda	• Mejor optimización para SEO gracias al Server-Side Rendering • Mayor velocidad de carga inicial (página ya renderizada) • Mayor seguridad: contraseñas, claves API y pagos se procesan en el backend • Menor dependencia del hardware del cliente • Centralización y consistencia de los datos
Desventajas	• Problemas de indexación SEO con contenido generado dinámicamente • Mayor vulnerabilidad, ya que el código fuente es visible • Carga inicial lenta al descargar todos los scripts • Dependencia del hardware del usuario • Inconsistencia entre navegadores	• Mayor costo y carga en el servidor por cada solicitud • Mayor latencia por los viajes de ida y vuelta al servidor • Recarga completa de pantalla entre páginas • Mayor tiempo para el primer byte (TTFB) • Desarrollo más complejo (infraestructura, bases de datos, balanceadores) • Dependencia total de la conexión a internet
Seguridad	• Protege el código (HTML, CSS y JavaScript) que corre en el navegador • Incluye validación de entradas, políticas CSP y monitoreo de scripts de terceros • Es inseguro por naturaleza, ya que el usuario tiene acceso total al código	• Es la barrera más fuerte, pues mantiene datos y lógica fuera del alcance del usuario • Incluye ocultamiento de datos sensibles, validación absoluta y control de acceso robusto • Mitigación de ataques comunes y auditoría de registros seguros
Ubicación de ejecución	• Se ejecuta en el dispositivo del usuario (navegador) • Ideal para interactividad inmediata, animaciones y validaciones rápidas	• Se ejecuta en el backend: un entorno de cómputo remoto separado del navegador • Corre en la infraestructura física o virtual del proveedor

